Real-world (make it feel alive with streaming) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `streaming_practice.py`: a normal call waits for the entire reply before showing you anything — for a long answer, that's several seconds of silence. Streaming prints each piece as it arrives, which is the difference between a chatbot that feels instant and one that feels stuck. **If not:** the Build Task's `stream_message()` would be the first time you ever saw a chunk-by-chunk response, with no smaller version to check your understanding against.

Solution

```
# streaming_practice.py
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

def stream_reply(history: list) -> str:
    # why: returns the full text once streaming finishes, so this can plug
    # straight into the same history list the memory exercise already builds.
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
        stream=True,
    )

    full_reply = ""
    for chunk in stream:
        # how: each chunk carries one small piece of the reply in
        # .delta.content — None on chunks that carry no new text.
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
            full_reply = full_reply + piece

    print()
```

```
    return full_reply

def main() -> None:
    history = [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain a REST API in 3 sentences."},
    ]
    reply = stream_reply(history)
    history.append({"role": "assistant", "content": reply})

if __name__ == "__main__":
    main()
```

Expected output: the reply printed piece by piece as it streams in, and `history` now correctly ends with the assistant's full reply appended.